Lab 2 - Chroot Jail

2026-11-04

Table of contents

1 Introduction **1**

2 A Simple Jail with Bash **1**

3 A Robust Jail **4**

4 Testing **9**

5 Going Further (Optional) **10**
5.1 Chroot vs. Namespaces 10
5.2 An SFTP-Only Jail 11

6 Cleanup **12**

1 Introduction

A chroot jail is a security mechanism that creates an isolated environment (a jail) for a running process and its children where the process cannot access files or commands outside of the jail. Reasons for creating such an environment include providing limited resources for FTP and SSH users along with creating a clean environment for testing software.

2 A Simple Jail with Bash

First we setup a simple jail that allows the user to execute specific bash commands we allow.

1. Become the root user.

```
sudo -i
```

2. Check your machine's architecture, since some of the commands below differ between ARM and x86-64 machines.

```
uname -m
```

   If it prints `aarch64`, follow the **ARM** commands. If it prints `x86_64`, follow the **x86-64** commands.

3. Create the directory for the jail.

```
mkdir /var/jail
```

4. Navigate to the jail.

```
cd /var/jail
```

5. Everything we want the user to access must be in the jail. We will want them to have access to the bash interpreter. Let's check the dependencies of the bash interpreter with the `ldd` command so that we can determine what to add to the jail.

```
ldd /bin/bash
```

   The output should be similar to the following depending on your setup:

```
linux-vdso.so.1
libtinfo.so.6 => /lib/aarch64-linux-gnu/libtinfo.so.6
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6
/lib/ld-linux-aarch64.so.1
```

   or

```
linux-vdso.so.1
libtinfo.so.6 => /lib/x86_64-linux-gnu/libtinfo.so.6
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2
```

6. While in the /var/jail, create directories that mimic the outputs above. Only create the appropriate directories for your machine.

```
# ARM Machines:
mkdir bin lib
```

```
# x86-64 Machines:
mkdir bin lib lib64
```

7. Copy the bash dependencies and bash into the appropriate directories

```
# ARM Machines:
cp /lib/aarch64-linux-gnu/libtinfo.so.6 lib
cp /lib/aarch64-linux-gnu/libc.so.6 lib
cp /lib/ld-linux-aarch64.so.1 lib
cp /bin/bash bin
```

```
# x86-64 Machines:
cp /lib/x86_64-linux-gnu/libtinfo.so.6 lib
cp /lib/x86_64-linux-gnu/libc.so.6 lib
cp /lib64/ld-linux-x86-64.so.2 lib64
cp /bin/bash bin
```

8. Add the ls command to the jail.

   i. Determine the dependencies

```
ldd /bin/ls
```

   ii. Copy the items you haven't already copied over. The exact libraries depend on your distribution and version, so copy **whatever `ldd` lists** that is not already in the jail. The commands below show the most common result; if your `ldd` output lists a library not shown here, copy it into `lib` the same way, and if `cp` reports that one of these does not exist, skip it.

```
# ARM Machines:
cp /lib/aarch64-linux-gnu/libselinux.so.1 lib
cp /lib/aarch64-linux-gnu/libpcre2-8.so.0 lib
cp /bin/ls bin
```

```
# x86-64 Machines:
cp /lib/x86_64-linux-gnu/libselinux.so.1 lib
cp /lib/x86_64-linux-gnu/libpcre2-8.so.0 lib
cp /bin/ls bin
```

9. Now go to jail

```
chroot /var/jail
```

10. See if you can use `ls` to view the contents.

11. Bash built-ins such as `pwd` and `cd` should work. Test each out.
12. Try to run commands that aren't any of the built-in bash commands. They should not work.

    ```
    touch some-file
    mkdir some-dir
    nano
    vim
    sudo apt list
    ```

    You should get error messages on each of those.
13. Exit the jail with `ctrl + d` or `exit`.

3 A Robust Jail

Adding everything we want to the jail manually can be time consuming. In this section, we provide a script that you can use to add everything we will need to the jail.

1. Make sure you are the root user. Run the following if you are not.

    ```
    sudo -i
    ```

2. Navigate home.

    ```
    cd ~
    ```

3. Remove the simple jail, since the script will recreate it.

    ```
    rm -rf /var/jail
    ```

4. Create a file named `jailsetup.sh` and place the following bash script in it. Note that this script works regardless of your machine's architecture. The script can also be found on Canvas.

    ```
    #!/bin/bash
    CHROOT='/var/jail'
    mkdir -p "$CHROOT"

    # loop over the args provided on execution of this file
    # these args are the files we want available in the jail
    for file in "$@";
    do
        # file -L follows symlinks (e.g., /usr/bin/vim ->
        ↪ /etc/alternatives/vim)
    ```

```
    # so that symlinked programs are still detected as ELF executables
    if [ -x "$file" ] && file -L "$file" | grep -q 'ELF'
    then
    # For executable files:
        ldd "$file" |      # get the shared libraries for the file
        awk '{print $3}' | # extract 3rd column (the library filepath)
        grep -E '^/' |     # keep absolute paths (lines starting with /)
        sort -u |          # remove duplicates
        while read -r lib;
        do
           # copy the library and preserve its directory structure
           cp --parents "$lib" "$CHROOT"
        done
        # copy the file and preserve its directory structure
        cp --parents "$file" "$CHROOT"
    else
    # for non-executable files
    # just copy the file and preserve its directory structure
        cp --parents "$file" "$CHROOT"
    fi
done

# Copy the dynamic linker for the architecture of your machine.

# for aarch64 (arm)
if [ -f /lib/ld-linux-aarch64.so.1 ]; then
    cp --parents /lib/ld-linux-aarch64.so.1 "$CHROOT"
fi

# for amd64 (x86-64)
if [ -f /lib64/ld-linux-x86-64.so.2 ]; then
    cp --parents /lib64/ld-linux-x86-64.so.2 "$CHROOT"
fi

# for i386 (x86-32)
if [ -f /lib/ld-linux.so.2 ]; then
    cp --parents /lib/ld-linux.so.2 "$CHROOT"
fi

echo "Ready. Enter the jail with: chroot $CHROOT"
```

5. Give jailsetup.sh executable permissions.

```
chmod +x jailsetup.sh
```

6. Run jailsetup.sh with the following provided input to setup the jail. Modifying the inputs will allow for more or less capability.

```
./jailsetup.sh  /bin/{ls,cat,rm,mkdir,bash} /usr/bin/{vim,nano,whoami}
 ↪  /etc/hosts
```

   Note that if `vim` or `nano` is not installed on your machine, `cp` will report that the file does not exist. Either install it first or remove it from the list.

   Also note that we do **not** add `scp`. The `scp` program works by launching `ssh`, which is not in the jail, so it would not work from inside. Copying files *into* a jail from outside is handled differently; see the optional SFTP section at the end of this lab.

7. Copy the terminal definitions (terminfo) into the jail. Full-screen programs such as `vim` and `nano` use these to know how to draw on your terminal. Without them, they will fail with an error such as "Error opening terminal". Depending on your release, the definitions are split between `/lib/terminfo` and `/usr/share/terminfo`, so copy both (if one of them does not exist, `cp` will say so; that is fine).

```
cp -r --parents /lib/terminfo /var/jail
cp -r --parents /usr/share/terminfo /var/jail
```

8. Add some useful special files.

   i. Create the device folder for our special nodes.

```
mkdir /var/jail/dev
```

   ii. Create the /dev/null node for discarding output.

```
mknod -m 0666 /var/jail/dev/null c 1 3
```

   iii. Create the /dev/random node for random value generation.

```
mknod -m 0666 /var/jail/dev/random c 1 8
```

   iv. Create the /dev/urandom node for pseudo random value generation.

```
mknod -m 0666 /var/jail/dev/urandom c 1 9
```

   v. Create the /dev/zero node, which produces an endless stream of zero bytes.

```
mknod -m 0666 /var/jail/dev/zero c 1 5
```

vi. Create the /dev/tty node, which refers to the current terminal. The OpenSSH documentation lists `null`, `zero`, and `tty` among the device nodes an interactive chrooted session needs.

```
mknod -m 0666 /var/jail/dev/tty c 5 0
```

9. Now we add a new user to the system to test everything out.

   i. Create a new user group called `chrootjail`

```
groupadd chrootjail
```

   ii. Create a new user named `newuser`. Note that you will be prompted for a password and other information about the new user. You can leave the extra information blank.

```
adduser newuser
```

   iii. Add `newuser` to the chrootjail group.

```
adduser newuser chrootjail
```

10. Confirm the user was added by listing out the files and folders at `/home`. You should see a folder for the newuser.

11. Add the new user's home directory to the jail.

```
mkdir -p /var/jail/home/newuser
```

12. Copy the default shell configuration files (such as `.bashrc` and `.profile`) from `/etc/skel` into the user's home directory. The `-T` flag copies the *contents* of `/etc/skel` rather than the folder itself.

```
cp -rT /etc/skel /var/jail/home/newuser
```

13. Make the new user the owner of the new user's home directory.

```
chown -R newuser:newuser /var/jail/home/newuser
```

14. Create the user and group files inside the jail, keeping only the lines for `root`, `newuser`, and the `chrootjail` group. The `grep -E` pattern matches lines that *start with* one of those names followed by a colon.

```
mkdir -p /var/jail/etc
grep -E '^(root|newuser):' /etc/passwd > /var/jail/etc/passwd
grep -E '^(root|newuser|chrootjail):' /etc/group > /var/jail/etc/group
```

View both files with `cat` to confirm they contain only those lines. Notice that we did **not** copy `/etc/shadow`, which holds the password hashes. The jail has no need for it, since sshd checks the password before entering the jail.

15. If it isn't already installed, install `openssh-server`.

```
apt install openssh-server
```

16. Alter the ssh configuration to jail users in the chroot jail group.

    i. Create a new file named `/etc/ssh/sshd_config.d/chrootjail.conf` with vim or nano. The main `sshd_config` file automatically includes every `.conf` file in this folder, and keeping our changes in their own file makes them easy to remove later.

    ii. Add the following to the file.

```
Match group chrootjail
    ChrootDirectory /var/jail/
    X11Forwarding no
    AllowTcpForwarding no
```

    iii. Save and close the file.

> **Where Match Blocks Go**
>
> Every line after a `Match` line applies only to the users that match, until the next `Match` line or the end of the file. If you instead add this to `/etc/ssh/sshd_config` directly (for example, if your system has no `sshd_config.d` folder or the file does not contain an `Include` line for it), it must be placed at the very **end** of the file.

    iv. Check that the jail directory is owned by root and not writable by anyone else. sshd requires this for the `ChrootDirectory` and **every** directory above it (`/var` and `/`). If this is not the case, logins will fail with a "bad ownership or modes for chroot directory" error in the logs.

```
ls -ld / /var /var/jail
```

Each line should show `root root` as the owner and group, and permissions of `drwxr-xr-x` (no `w` for group or others). If `/var/jail` does not, fix it with `chown root:root /var/jail` and `chmod 755 /var/jail`.

v. Test the ssh configuration for errors. If there are no errors, this command prints nothing.

```
sshd -t
```

vi. Restart the ssh server.

```
systemctl restart ssh
```

17. Login via ssh with `newuser`.

```
ssh newuser@localhost
```

4 Testing

Once logged in, try the following to test out the chroot jail.

1. Navigate to the root directory and view the contents.

```
cd /
ls
```

You should only see the directories that were added to the chroot jail. Compare this with what is normally found at the root directory.

2. Try to navigate higher.

```
cd ..
```

You should see that you cannot navigate higher.

3. Try printing the working directory to see if it shows any information about the system above the chroot jail's root directory.

```
pwd
```

You should see that it doesn't.

4. Try switching to the root user.

```
sudo -i
```

You should see that the sudo command doesn't exist in the chroot jail since we didn't add it.

5. View the list of users.

   ```
   cat /etc/passwd
   ```

   You should only see `root` and `newuser`. The jailed user cannot discover the other accounts on the system, since the real `/etc/passwd` is outside the jail.

6. Test anything else you would like to. If you would like to add any additional commands to the jail, simply run the `jailsetup.sh` script again with the appropriate arguments. For example, if you want to add `touch` you would run `./jailsetup.sh /bin/touch`.

7. Once done, disconnect from the ssh server and proceed to the cleanup section of this document.

> ⚠ Chroot Is Not a Complete Security Boundary
>
> The jail only works because `newuser` is not root. A process running as **root** inside a chroot jail can escape it (for example, by calling `chroot()` again on a subdirectory and then moving up with `..`). This is why we never add `sudo`, `su`, or any other *setuid* program (one that runs with its owner's privileges, such as `passwd`) to the jail. Chroot also only isolates the file system: jailed processes still share the process list, network, and kernel with the rest of the system. Containers (e.g., Docker) build on Linux namespaces to provide much stronger isolation.

5 Going Further (Optional)

5.1 Chroot vs. Namespaces

This section shows the "only the file system is isolated" limitation directly. Run these as root on the host (not logged in as `newuser`), from root's home directory where `jailsetup.sh` is (`cd ~`).

1. Add `ps` to the jail, mount the kernel's process list (`/proc`) inside it, and list processes from inside the jail.

   ```
   ./jailsetup.sh /usr/bin/ps
   mkdir -p /var/jail/proc
   mount -t proc proc /var/jail/proc
   chroot /var/jail /usr/bin/ps aux | head -n 15
   ```

   Even though the command runs inside the jail, you see processes from the **whole system**. A jailed process with access to `/proc` can see, and if permissions allow, signal, processes outside its jail.

2. Unmount `/proc` from the jail. Do not skip this, or `rm -rf /var/jail` in the cleanup will try to delete through it.

```
umount /var/jail/proc
```

3. Now compare with a PID *namespace*, which gives a process its own private process list.

```
unshare --pid --fork --mount-proc bash
ps aux
```

   This time `ps aux` lists only two processes (`bash` and `ps` itself), and `bash` is process 1. The rest of the system is invisible. Type `exit` to leave the namespace.

Containers such as Docker and Podman combine a chroot-style file system with namespaces like this one (plus network, mount, and user namespaces, and cgroups for resource limits).

5.2 An SFTP-Only Jail

The most common real-world use of `ChrootDirectory` is giving users a place to upload and download files with SFTP, with no shell at all. OpenSSH has a built-in SFTP server (`internal-sftp`) that runs inside the sshd process, so the jail does not need any programs or libraries.

1. Edit `/etc/ssh/sshd_config.d/chrootjail.conf` and add a `ForceCommand` line to the Match block:

```
Match group chrootjail
    ChrootDirectory /var/jail/
    ForceCommand internal-sftp
    X11Forwarding no
    AllowTcpForwarding no
```

2. Check the configuration and restart the ssh server.

```
sshd -t
systemctl restart ssh
```

3. From a normal (non-root) terminal, connect with `sftp` and try it out. The user can only reach files inside the jail, and `ssh newuser@localhost` will no longer give a shell.

```
echo "hello from outside" > somefile.txt
sftp newuser@localhost
sftp> cd /home/newuser
sftp> put somefile.txt
```

```
sftp> ls
sftp> exit
```

Remember that `newuser` can only write to `/home/newuser`, since the jail root itself must be owned by root and not writable by others. `scp` also works now, because modern `scp` uses the SFTP protocol.

The Cleanup section removes this configuration file, so no extra cleanup is needed.

6 Cleanup

To reset everything back to how it was prior, complete the following steps. Be sure you are not still connected through ssh with the newuser account.

1. Switch to the root user.

```
sudo -i
```

2. Remove the ssh configuration we added, then restart the ssh server so that the change takes effect.

```
rm /etc/ssh/sshd_config.d/chrootjail.conf
sshd -t
systemctl restart ssh
```

If you added the `Match group chrootjail` lines directly to `/etc/ssh/sshd_config` instead, open that file, remove those four lines, save it, and then run `sshd -t` and `systemctl restart ssh`.

3. Remove the jail. The `umount` line makes sure `/proc` is not still mounted inside it (from the optional section); if it was never mounted, the error message can be ignored.

```
umount /var/jail/proc
rm -rf /var/jail
```

4. Remove the new user.

```
deluser --remove-home newuser
```

5. Remove the chrootjail group.

```
delgroup --only-if-empty chrootjail
```